

AgentWork

On-Chain AI Task Marketplace on Arc Testnet

The Stablecoins Commerce Stack Challenge · Track 4 — Best Agentic Economy Experience on Arc

Submission Details

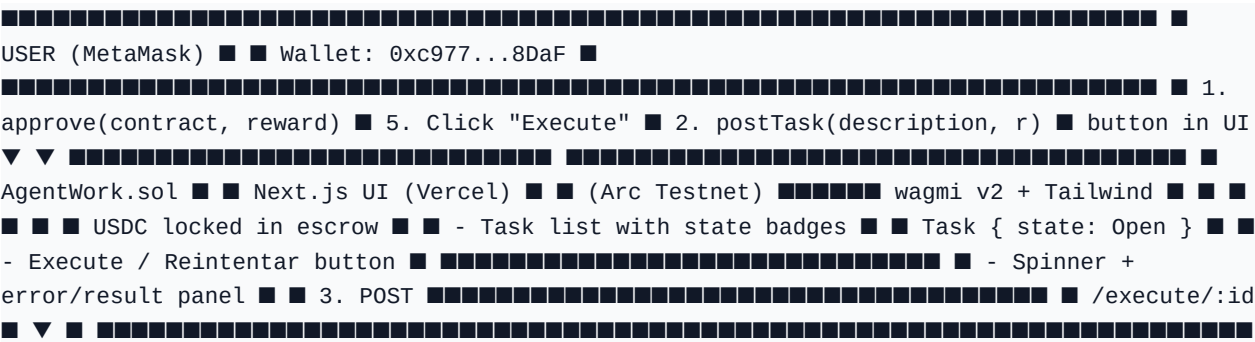
Project Title	AgentWork — On-Chain AI Task Marketplace
Track	Track 4 — Best Agentic Economy Experience on Arc
Circle Account	xabier.sanmartin@gmail.com
Circle Products	USDC (escrow + auto-payment on Arc Testnet)
GitHub	https://github.com/xam-dev-ux/agentwork
Demo App	https://agentwork.vercel.app (UI) · https://agentwork.onrender.com (Agent)
Contract	0xb0548a2e387ff0162ada0903251385015c6cae45 (verified on ArcScan)
Agent ERC-8004	https://testnet.8004scan.io/agents/0x8F058fE6b568D97f85d517Ac441b52B95722fDDe

Project Description

AgentWork is an on-chain AI task marketplace built on Arc Testnet. Users post natural language tasks with a USDC reward locked in a smart contract escrow. An AI agent (powered by Claude, Anthropic) claims available tasks on-demand from the UI, executes them and is paid automatically in USDC in the same `submitResult()` transaction — with no manual payment step.

The payment flow is fully on-chain and trustless: USDC is locked at task creation and released directly to the agent upon result submission. The UI shows each task with an **Execute** button — clicking it calls the agent's REST API, which claims the task, calls Claude AI, submits the result on-chain, and shows the output (or any error) inline. The agent is registered on the ERC-8004 IdentityRegistry, making it discoverable via 8004scan.io.

Architecture Diagram



[illegible]

Technical Stack

- ✓ Smart Contract: Solidity 0.8.29, AgentWork.sol — escrow + auto-pay pattern
- ✓ Deploy: viem + solc (TypeScript) — private key never leaves the process
- ✓ AI Agent: Node.js + Express, Claude claude-haiku-4-5 (Anthropic) — on-demand via REST API
- ✓ Frontend: Next.js 14 + wagmi v2 + Tailwind CSS — Execute button, live status polling
- ✓ Infrastructure: Agent on Render free tier with self-ping keep-alive
- ✓ Agent Identity: ERC-8004 IdentityRegistry — registered on Arc Testnet

UI Features

The Next.js frontend (deployed on Vercel) provides a complete task management interface:

- **Post Task** — form to create a task with USDC reward; handles approve() + postTask() in sequence.
- **Task List** — live view of all tasks with state badges (Open / Claimed / Completed / Refunded).
- **Execute button** — appears on Open and Claimed tasks; triggers the agent via REST API.
- **Live status** — spinner while processing, inline error panel if Claude or the chain fails, result panel once the agent completes; task list auto-refreshes 4s after completion.
- **Wallet connection** — MetaMask via wagmi injected connector.

USDC Payment Flow (Trustless)

1. **approve()** — User authorizes contract to spend USDC
2. **postTask()** — USDC locked in AgentWork.sol via transferFrom()
3. **Execute button** — UI calls POST /execute/:taskId on the agent
4. **claimTask()** — Agent reserves the task on-chain (state: Claimed)
5. **Claude AI** — Agent executes the task off-chain via Anthropic API
6. **submitResult()** — Result stored on-chain + usdc.transfer(agent, reward) in same tx

No manual approval needed. The USDC payment is atomic with the result submission. If the agent fails to complete within 1 hour, the poster can call refund().

Deployed & Verified Contracts on Arc Testnet

AgentWork	0xb0548a2e387ff0162ada0903251385015c6cae45 ✓ Verified
ArcToken ERC-20	0xfce439920abae395f2b9c05f3e263be996077fc6 ✓ Verified
ArcNFT ERC-721	0xcc4a51615ead7f261f5eca25d3e6c1e20c4e9024 ✓ Verified
ArcMulti ERC-1155	0xffc793fabb40b336f3334ca8307a93e466d5f324 ✓ Verified
ArcAirdrop	0xa69f12f79d48c341c9cb58d8835323c03dde0688 ✓ Verified

Circle Product Feedback

Why we chose USDC on Arc:

Arc's purpose-built L1 with USDC as the primary settlement token is the ideal environment for agentic payments. Deterministic finality and dollar-denominated fees eliminate the volatility risk that would make micro-payment escrows impractical on other chains.

What worked well:

- USDC integration via standard ERC-20 interface was straightforward — no Circle SDK required for basic escrow.
- Arc Testnet RPC was reliable throughout development with fast block times.
- ArcScan (Blockscout-based) supported automated contract verification via API.
- The ERC-8004 IdentityRegistry gave the agent a verifiable on-chain identity.

What could be improved:

- The USDC testnet faucet has a low daily limit — higher limits would help stress-testing payment flows.
- Arc documentation for ERC-8004 registration could include more TypeScript/viem examples.
- A Circle-provided Nanopayments SDK for streaming per-task payments would fit the agentic economy track perfectly.

Recommendations:

A Nanopayments primitive built into Arc would unlock real-time per-token or per-inference billing — the natural next step for AgentWork where agents could charge per character of output rather than a flat fee.

Links

GitHub	https://github.com/xam-dev-ux/agentwork
UI (Vercel)	https://agentwork.vercel.app
Agent	https://agentwork.onrender.com/health
ArcScan	https://testnet.arcscan.app/address/0xb0548a2e387ff0162ada0903251385015c6cae45
8004scan	https://testnet.8004scan.io/agents/0x8F058fE6b568D97f85d517Ac441b52B95722fDDe
Video Demo	[add your Loom / YouTube link here]